شده و از وضعیت ذخیره شده از سر گرفته می‌شود در صورتی که کاربر باز هم بـه برنامـه سرک بکشد.

۳-۴ ارتباط بین پردازه‌ای

پردازه‌های در حال اجرای سیستم‌عامل می‌توانند پردازه‌های **مستقل** و یا پردازه‌های **همکار**[1] باشند. یک پردازه ***مستقل*** است اگر هیچ داده‌ی مشترکی با سایر پردازه‌هایی که در سیستم اجرا می‌شوند نداشته باشد. پردازه، ***همکار*** نامیده می‌شود اگر روی اجرای پردازه‌های دیگر تأثیر گذاشته یا از اجرای پردازه‌های دیگر متأثر شود. بدیهی است که پردازه با دیگر پردازه‌هایی که اشتراک داده دارد، همکار است.

به دلایل متعددی می‌بایست محیطی فراهم شود که پردازه‌ها بتوانند با همدیگر امکان همکاری داشته باشند که عبارتند از:

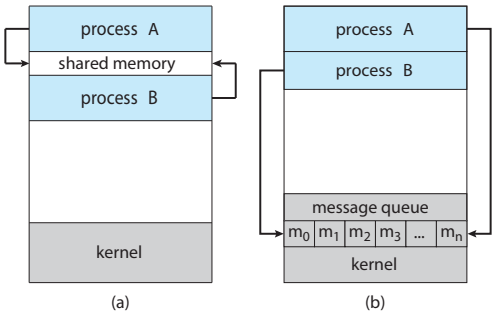
- **اشتراک اطلاعات**: از آنجایی که چندین برنامـه ممکـن اسـت بـه یـک سـری اطلاعـات یکسان (برای نمونه، کپی‌کردن و چسباندن[2]) علاقه‌مند باشند، بایستی محیطی فـراهم کنیم تا دسترسی همروند این قبیل اطلاعات امکان‌پذیر باشد.
- **تسریع محاسبات**: اگر تمایل داریم که کار خاصی سریع‌تر اجرا شـود، بایسـتی آن را به کارهای کوچکتری تقسیم کنیم که هر یک از آنها بتواننـد مـوازی بـا سـایر کارهـا اجرا شوند. توجه داشته باشید که این تسریع فقط در کامپیوتری قابل حصول اسـت که هسته‌های پردازشی متعددی دارد.
- **پیمانه‌سازی**[3]: شاید بخواهیم سیستم را با تقسیم کارکردهای سیستم به پردازه‌ها یـا نخ‌های مجزا به شیوه‌ی پیمانه‌ای بنا کنیم، همان‌گونه که در فصل ۲ بحث کردیم.

پردازه‌های همکار به یک راهکار **ارتباط بین پردازه‌ای**[4] (**IPC**) نیاز دارند تا بتوانند داده‌هـا را مبادله کنند—یعنی، داده‌ها را به همدیگر ارسال کرده و داده‌ها را از همدیگر دریافت کننـد. دو مدل اساسی جهت ارتباط بین پردازه‌ها وجـود دارنـد کـه عبارتنـد از: **حافظـه‌ی مشـترک** و **عبور پیغام**. در مدل حافظه‌ی مشترک، یک ناحیه از حافظه که توسط پـردازه‌هـای همکـار بـه اشتراک گذاشته می‌شود، برپا می‌شود. پس از آن پردازه‌ها مـی‌تواننـد بـا خوانـدن و نوشـتن داده‌ها در ناحیه‌ی مشترک به تبادل اطلاعات بپردازند. در مدل عبور پیغـام، ارتبـاط از طریـق پیغام‌هایی که میان پردازه‌های همکار مبادله می‌شوند صورت می‌گیـرد. دو مـدل ارتبـاطی در شکل ۳-۱۱ مقایسه می‌شوند.

هر دو مدلی که عنوان شدند در سیستم‌عامل‌ها رواج دارند و بسیاری از سیستم‌ها هر دو مدل را پیاده‌سازی می‌کنند. عبور پیغام در مبادله‌ی حجم کمتری از داده‌ها مفید است، چون نیازی نیست که از برخوردها اجتناب کنیم. پیاده‌سازی عبور پیغام در سیستم توزیع‌شده به نسبت

1. Cooperating processes
2. Pasting
3. Modularity
4. Interprocess communication

حافظه‌ی مشترک، ساده‌تر هم هست. (هرچند سیستم‌هایی هستند که حافظه‌ی مشترک توزیع‌شده را فراهم می‌کنند که در این کتاب مد نظر ما نیستند.) حافظه‌ی مشترک می‌تواند سریع‌تر از عبور پیغام باشد، چون سیستم‌های عبور پیغام نوعاً با استفاده از فراخوانی‌های سیستمی پیاده‌سازی می‌شوند و لذا وقت‌گیری بیشتر کار مداخله‌ی هسته را لازم دارند. در سیستم‌های حافظه‌ی مشترک، فراخوانی‌های سیستمی فقط برای برپایی نواحی مشترک حافظه لازم می‌شوند. همین که حافظه‌ی مشترک برپا شد، تمامی دسترسی‌ها به عنوان دسترسی‌های معمول حافظه تلقی شده و هیچ‌گونه نیازی به یاری هسته پیش نخواهد آمد.



شکل ۳-۱۱ مدل‌های ارتباط. (*a*) تبادل پیغام. (*b*) حافظه‌ی مشترک.

معماری چندپردازه‌ای-مرورگر کروم

بسیاری از وب‌سایت‌ها دارای محتوای فعالی چون JavaScript، Flash و HTML5 هستند تا یک تجربه‌ی وب‌گردی غنی و پویا را فراهم کنند. متأسفانه، این برنامه‌های وب ممکن است حاوی ایرادهای نرم‌افزاری باشند که می‌تواند باعث زمان‌های پاسخ طولانی شده و حتی منجر به از کار افتادن مرورگر وب شود. این موضوع در یک مرورگر وب که فقط محتوای یک وب‌سایت را نمایش می‌دهد مشکل بزرگی نیست. اما اکثر مرورگرهای وب کنونی مرور زبانه‌ای[1] را فراهم می‌کنند که به یک نمونه‌ی منفرد از برنامه‌ی مرورگر وب اجازه می‌دهند تا چندین وب‌سایت را همزمان، که هر سایت در یک زبانه‌ی مجزاست، باز کند. کاربر به منظور جابه‌جایی میان سایت‌های متفاوت فقط لازم دارد که روی زبانه‌ی مناسب کلیک کند. این وضعیت در زیر به تصویر کشیده می‌شود:



مسأله‌ای که این رویکرد دارد این است که اگر یک برنامه‌ی وب در هر کدام از زبانه‌ها خراب شود، کل پردازه-اعم از سایر زبانه‌ها که وب‌سایت‌های دیگری را نمایش می‌دهند-نیز با شکست مواجه می‌شود.

1. Tabbed browsing

مرورگر وب کروم[1] گوگل طراحی شد تا این موضوع را با استفاده از معماری چندپردازه‌ای حل کند. کروم سه نوع پردازه‌ی متفاوت را شناسایی می‌کند که عبارتند از: browser، renderes و plug-ins.

- پردازه‌ی **browser** مسؤول مدیریت رابط کاربر، در کنار I/O دیسک و شبکه است. موقعی که کروم شروع به کار می‌کند، یک پردازه‌ی browser جدید ایجاد می‌شود. تنها یک پردازه‌ی browser ایجاد می‌شود.
- پردازه‌های **renderer** شامل منطق پرداخت صفحات وب هستند. بنابراین، شامل منطقی برای راه‌اندازی HTML، Javascript، تصاویر و از این دست می‌باشند. به عنوان یک قاعده‌ی کلی، برای هر یک از وب‌سایت‌هایی که در یک زبانه‌ی جدید باز می‌شوند یک پردازه‌ی renderer جدید ایجاد می‌شود و لذا چندین پردازه‌ی renderer ممکن است همزمان فعال باشند.
- یک پردازه‌ی **plug-in** برای هر نوع plug-in در حال استفاده (از قبیل Flash یا QuickTime) ایجاد می‌شود. پردازه‌های plug-in حاوی کد plug-in، در کنار کد اضافی هستند که plug-in را قادر می‌سازد تا با پردازه‌های renderer مرتبط و پردازه‌ی browser ارتباط برقرار کند.

مزیت رویکرد چندپردازه‌ای این است که وب‌سایت‌ها به صورت جدا از همدیگر اجرا می‌شوند. اگر یک وب‌سایت از کار بیفتد، تنها پردازه‌ی renderer آن متأثر می‌شود؛ سایر پردازه‌ها آسیبی نمی‌بینند. به علاوه، پردازه‌های renderer در یک گودال شنی[2] اجرا می‌شوند، با این مضمون که دسترسی به I/O دیسک و شبکه محدود می‌شود طوری که آثار هرگونه بهره‌برداری امنیتی کمینه می‌گردد.

در بخش ۳-۵ و بخش ۳-۶، سیستم‌های حافظه‌ی مشترک و عبور پیغام را با جزئیات بیشتری بررسی می‌کنیم.

۳-۵ IPC در سیستم‌های حافظه-مشترک

ارتباط بین پردازه‌ها با استفاده از حافظه‌ی مشترک مستلزم آن است که پردازه‌های مرتبط، ناحیه‌ای را برای اشتراک حافظه برپا کنند. ناحیه‌ی مشترک حافظه معمولاً در فضای آدرس پردازه‌ای که سگمنت مشترک حافظه را ایجاد می‌کند قرار می‌گیرد. سایر پردازه‌هایی که تمایل به استفاده از این سگمنت مشترک حافظه را دارند بایستی آن را به فضای آدرس خود پیوند بزنند. به یاد دارید که سیستم‌عامل در اصل سعی دارد تا از دسترسی فضای آدرس یک پردازه توسط پردازه‌ای دیگر جلوگیری کند. لازمه‌ی اشتراک حافظه این است که دو یا چند پردازه با هم به توافق برسند تا این محدودیت را بر دارند. سپس می‌توانند اطلاعات را با خواندن و نوشتن داده‌ها در نواحی مشترک با همدیگر مبادله کنند. شکل داده‌ها و مکان آنها توسط این پردازه‌ها تعیین می‌شود و تحت کنترل سیستم‌عامل نیست. مسؤولیت تضمین این که پردازه‌ها همزمان اقدام به نوشتن در مکان یکسانی نخواهند کرد نیز بر عهده‌ی پردازه‌های مرتبط است.

بیایید برای روشن‌شدن مفهوم پردازه‌های همکار، مسأله‌ی تولیدکننده-مصرف‌کننده را مرور کنیم که یک تئوری مرسوم در رابطه با پردازه‌های همکار است. پردازه‌ی **تولیدکننده** اطلاعاتی تولید می‌کند که توسط پردازه‌ی **مصرف‌کننده** مصرف می‌شود. برای مثال، یک کامپایلر کد اسمبلی

1. Chrome
2. Sandbox

تولید می‌کند که توسط اسمبلر مصرف می‌شود. اسمبلر هم به نوبت خود ماژول‌های مقصد را تولید می‌کند که بارگذار مصرف می‌کند. مسأله‌ی تولیدکننده–مصرف‌کننده برای تئوری کارفرما–کارگزار هم نمونه‌ی عملی مفیدی فراهم می‌کند. در حالت کلی، هر کارگزار را تولیدکننده و هر کارفرما را یک مصرف‌کننده می‌پنداریم. برای مثال، کارگزار وب، محتوای وبی همچون فایل‌های HTML و تصاویری تولید می‌کند (یعنی، فراهم می‌کند) که توسط مرورگر وب، کارفرمای درخواست کننده‌ی آن منبع مصرف می‌شود (یعنی، می‌خواند).

یک راه‌حل مسأله‌ی تولیدکننده–مصرف‌کننده از حافظه‌ی مشترک استفاده می‌کند. برای این که بتوانیم اجرای همروند پردازه‌ی تولیدکننده و مصرف‌کننده را فراهم کنیم بایستی بافری از اقلام داده‌ها داشته باشیم که توسط تولیدکننده پر شده و توسط مصرف‌کننده خالی شود. این بافر در ناحیه‌ای از حافظه که توسط پردازه‌های تولیدکننده و مصرف‌کننده مشترک است قرار خواهد گرفت. تولیدکننده می‌تواند در حالی که مصرف‌کننده در حال مصرف قلم‌داده‌ی دیگری است قلم‌داده‌ی جدیدی را تولید کند. تولیدکننده و مصرف‌کننده می‌بایست به نحوی با هم همگام شوند تا مصرف‌کننده سعی نکند اقدام به مصرف قلم‌داده‌ای نماید که هنوز تولید نشده است.

برای این کار می‌توان از دو نوع بافر استفاده کرد. **بافر نامحدود**[1] که در عمل هیچ‌گونه محدودیتی روی اندازه‌ی بافر نمی‌گذارد. مصرف‌کننده ممکن است به ناچار منتظر قلم‌داده‌های جدیدی بماند، اما تولیدکننده همیشه می‌تواند قلم‌داده‌های جدیدی را تولید کند. **بافر محدود**[2]، فرض می‌کند که اندازه‌ی بافر ثابت است. در اینجا، اگر بافر خالی باشد مصرف‌کننده بایستی منتظر شود و در صورتی که بافر پر باشد، تولیدکننده به ناچار منتظر خواهد ماند.

بیایید از نزدیک ببینیم که چگونه بافر محدود، ارتباط بین‌پردازه‌ای را با استفاده از حافظه‌ی مشترک روشن می‌سازد. متغیرهای زیر در ناحیه‌ی مشترک حافظه‌های پردازه‌های تولیدکننده و مصرف‌کننده قرار دارند:

```
#define BUFFER_SIZE 10
typedef struct {
        ...
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

بافر مشترک با استفاده از دو اشاره‌گر منطقی in و out، به صورت یک آرایه‌ی چرخشی پیاده‌سازی می‌شود. متغیر in به مکان خالی بعدی بافر اشاره می‌کند؛ متغیر out نیز به اولین مکان پر بافر اشاره دارد. اگر in == out باشد، در این صورت بافر خالی است؛ بافر در صورتی پر خواهد بود که، (in + 1) % BUFFER_SIZE == out شود.

1. Unbounded buffer
2. Bounded buffer

کد پردازه‌ی تولیدکننده در شکل ۳-۱۲ و کد پردازه‌ی مصرف‌کننده در شکل ۳-۱۳ نشان داده می‌شود. پردازه‌ی تولیدکننده یک متغیر محلی به نام next_produced دارد که قلم‌داده‌ی جدید را در آن ذخیره می‌کند. پردازه‌ی مصرف‌کننده نیز یک متغیر محلی به نام next_consumed دارد که قلم‌داده‌ی مصرفی آن را ذخیره دارد.

```
item next_produced;
while (true) {
        /* produce an item in next_produced */
        while ((in + 1) % BUFFER_SIZE) == out)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
}
```

شکل ۳-۱۲ پردازه‌ی تولیدکننده با استفاده از حافظه‌ی مشترک.

```
item next_consumed;
while (true) {
        while (in == out)
                ; // do nothing

        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        /* consume the item in next_consumed */
}
```

شکل ۳-۱۳ پردازه‌ی مصرف‌کننده با استفاده از حافظه‌ی مشترک.

با این طرح، حداکثر به تعداد BUFFER_SIZE – 1 قلم‌داده را می‌توان همزمان در بافر ذخیره کرد. به عنوان یک تمرین، راه‌حلی ارائه بدهید که بتوان همزمان BUFFER_SIZE قلم‌داده را در بافر ذخیره کرد. در بخش ۳-۷-۱، API پوزیکس را برای روشن‌شدن مفهوم حافظه‌ی مشترک نشان می‌دهیم.

یک مورد نگران‌کننده که در اینجا مد نظر نیست، این است که امکان دارد شرایطی پیش بیاید که پردازه‌های تولیدکننده و مصرف‌کننده بخواهند بافر مشترک را توأمان به صورت همروند دستیابی کنند. در فصل ۶ و فصل ۷، چگونگی پیاده‌سازی کارآمد همگامی مابین پردازه‌های همکار را در یک محیط حافظه‌ی مشترک بحث می‌کنیم.

۳-۶ IPC در سیستم‌های عبور-پیغام

در بخش ۳-۵، نشان دادیم که چگونه پردازه‌های همکار می‌توانند با یک محیط حافظه‌ی مشترک با همدیگر ارتباط داشته باشند. لازمه‌ی این طرح آن است که این پردازه‌ها ناحیه‌ای از حافظه را به اشتراک بگذارند و کد دسترسی و دستکاری حافظه‌ی مشترک نیز آشکارا توسط برنامه‌نویس کاربردی نوشته شود. روش دیگر حصول این هدف توسط سیستم‌عامل آن است

که ابزاری برای پردازه‌های همکار فراهم کند تا بتوانند از طریق امکان عبور پیغام با همدیگر در ارتباط باشند.

عبور پیغام، راهکاری فراهم می‌کند که در آن پردازه‌ها می‌توانند با همدیگر ارتباط برقرار کرده و اعمال خود را چنان همگام سازند که نیازی به اشتراک فضای آدرس یکسان نداشته باشند. آن مخصوصاً در یک محیط توزیع‌شده که پردازه‌های مرتبط از کامپیوترهای متفاوت به یک شبکه متصل هستند کارآمدتر خواهد بود. برای مثال، برنامه‌ی **chat** اینترنتی می‌تواند طوری طراحی شود که افراد شرکت‌کننده با عبور پیغام با همدیگر در ارتباط باشند.

امکان عبور پیغام، حداقل دو عملیات را فراهم می‌کند که عبارتند از:

```
send(message)
```

و

```
receive(message)
```

پیغام‌های ارسالی پردازه می‌توانند در اندازه، ثابت و یا متغیر باشند. اگر بتوان تنها پیغام‌هایی با اندازه‌ی ثابت ارسال کرد، پیاده‌سازی آن در سطح سیستم سر راست‌تر خواهد بود. با این وجود، کار برنامه‌نویسی به خاطر این محدودیت با مشکلات بیشتری مواجه می‌شود. برعکس، پیاده‌سازی سطح سیستم پیغام‌های با اندازه‌ی متغیر پیچیدگی بیشتری دارد، اما کار برنامه‌نویسی ساده‌تر می‌شود. این یک نوع مصالحه‌ی رایجی است که در کل طراحی سیستم‌عامل به چشم می‌آید.

اگر پردازه‌های P و Q بخواهند در ارتباط باشند، بایستی پیغام‌هایی به همدیگر ارسال و از همدیگر دریافت کنند؛ لذا باید یک *خط ارتباطی*[1] بین آنها برقرار شود. این خط می‌تواند به روش‌های مختلف پیاده‌سازی شود. در اینجا، دغدغه‌ی پیاده‌سازی فیزیکی خط (نظیر حافظه‌ی مشترک، گذرگاه سخت‌افزاری یا شبکه که در فصل ۱۹ پوشش داده می‌شوند) را نداریم، اما به جای آن، روی پیاده‌سازی منطقی تمرکز می‌کنیم. در اینجا، چندین روش پیاده‌سازی منطقی خط ارتباطی و عملیات send()/receive() را مطرح می‌کنیم:

- ارتباط مستقیم یا غیرمستقیم
- ارتباط همگام یا ناهمگام
- بافرسازی خودکار یا آشکار

در ادامه، به ملاحظات مربوط به هر یک از این ویژگی‌ها می‌پردازیم.

۳-۶-۱ نام‌گذاری

پردازه‌هایی که می‌خواهند با هم در ارتباط باشند می‌بایست به نحوی به همدیگر رجوع کنند. آنها می‌توانند از ارتباط مستقیم و یا غیر مستقیم استفاده کنند.

در **ارتباط مستقیم** بایستی هر یک از پردازه‌ها، آشکارا طرف گیرنده یا فرستنده را در ارتباط نام ببرند. بدویات[2] send() و receive() این طرح به صورت زیر تعریف می‌شوند:

1. Communication link
2. Primitives

- send(P, message)یک پیغام به پردازه‌ی P ارسال کن.
- receive(Q, message)یک پیغام از پردازه‌ی Q دریافت کن.

خواص خط ارتباطی این طرح عبارتند از:

- یک خط ارتباطی به طور خودکار، میان هر زوج از پردازه‌ها که خواهان برقراری ارتباط باشند، برپا می‌شود. پردازه‌ها برای برقراری ارتباط فقط لازم دارند که شناسه‌ی همدیگر را بدانند.
- هر خط ارتباطی دقیقاً به دو پردازه مربوط می‌شود.
- میان هر زوج از پردازه‌ها دقیقاً یک خط ارتباطی وجود دارد.

این طرح، یک *تقارن* را در آدرس‌دهی نشان می‌دهد؛ یعنی، پردازه‌های فرستنده و گیرنده در ارتباط، هر دو بایستی طرف دیگر را نام ببرند. گونه‌ی دیگری از این طرح وجود دارد که از آدرس‌دهی *نامتقارن* استفاده می‌کند. در اینجا، فقط فرستنده است که نام گیرنده را مشخص می‌کند؛ الزامی نیست که گیرنده نیز نام فرستنده را مشخص نماید. بدویات ()send و ()receive این طرح به صورت زیر تعریف می‌شوند:

- send(P, message)یک پیغام به پردازه‌ی P ارسال کن.
- receive(id, message)یک پیغام از هر پردازه‌ای که شد، دریافت کن. متغیر id نام پردازه‌ای انتخاب می‌شود که ارتباط با آن صورت می‌گیرد.

عیبی که هر دوی این طرح‌ها (متقارن و نامتقارن) دارند، ماژولاریتی محدود حاصل از تعاریف پردازه‌هاست. تغییر شناسه‌ی یک پردازه ممکن است بازبینی و بررسی تعاریف سایر پردازه‌ها را نیز لازم داشته باشد. بایستی تمامی مراجعات قبلی آن شناسه پیدا شده و به شناسه‌ی جدید اصلاح شوند. در حالت کلی، این قبیل تکنیک‌های **کدنویسی سخت**[1] که لازم دارند شناسه‌ها آشکارا بیان شوند، در مقایسه با تکنیک‌های غیرمستقیم موجود که در ادامه مطرح می‌شوند از مطلوبیت کمتری برخوردار هستند.

در *ارتباط غیرمستقیم*، پیغام‌ها به **صندوق‌های پستی** یا **درگاه‌ها**[2] ارسال و یا از آنها دریافت می‌شوند. صندوق پستی را می‌توان به صورت مدل شیئی تصور کرد که پردازه‌ها می‌توانند در آن پیغام گذاشته و یا از آن پیغام بر دارند. هر صندوق پستی، یک شناسه‌ی واحد دارد. برای مثال، صف‌های پیغام پوزیکس برای شناسایی یک صندوق پستی از یک مقدار صحیح استفاده می‌کنند. پردازه می‌تواند از طریق یک سری شماره صندوق متفاوت با پردازه‌ای دیگر در ارتباط باشد، اما دو پردازه فقط در صورتی قادر به ارتباطند که صندوق پستی مشترکی داشته باشند. بدویات ()send و ()receive این طرح به صورت زیر تعریف می‌شوند:

- send(A, message)یک پیغام به صندوق پستی A ارسال کن.
- receive(A, message)یک پیغام از صندوق پستی A دریافت کن.

خط ارتباطی این طرح دارای خواص زیر است:

- تنها در صورتی خط ارتباطی میان یک زوج از پردازه‌ها برپا می‌شود که هر دو

1. Hard-coding
2. Ports

عضو زوج، صندوق پستی مشترکی داشته باشند.

- یک خط ارتباطی می‌تواند به بیش از دو پردازه مربوط شود.
- یک سری خط ارتباطی متفاوت ممکن است میان هر زوج از پردازه‌های مرتبط باشند که هر خط متناظر یک صندوق پستی است.

اکنون فرض کنید که پردازه‌های P_1، P_2 و P_3، همگی صندوق پستی A را به اشتراک بگیرند. پردازه‌ی P_1 یک پیغام به A می‌فرستد، در حالی که هر دوی پردازه‌های P_2 و P_3، عملیات ()receive را از A اجرا می‌کنند. کدام یک از پردازه‌ها پیغام ارسالی P_1 را دریافت خواهد کرد؟ جواب این سؤال به یکی از متدهای زیر که انتخاب می‌کنیم بستگی دارد:

- اجازه بدهید که یک خط ارتباطی، حداکثر به دو پردازه مربوط شود.
- اجازه بدهید که در هر لحظه، حداکثر یک پردازه بتواند عملیات ()receive را اجرا کند.
- اجازه بدهید که سیستم به صورت دلخواه تعیین کند که کدام پردازه پیغام را دریافت خواهد کرد (یعنی، P2 و یا P3، و نه هر دو). سیستم می‌تواند یک الگوریتم هم برای گزینش پردازه‌ی گیرنده‌ی پیغام تعریف کند (برای مثال، *نوبت گردشی*، که پردازه‌ها برای دریافت پیغام‌ها نوبت می‌گیرند). سیستم می‌تواند گیرنده را برای فرستنده مشخص کند.

مالکیت یک صندوق‌پستی می‌تواند در اختیار یک پردازه و یا سیستم‌عامل باشد. در صورتی که مالکیت صندوق‌پستی در اختیار یک پردازه باشد (یعنی، صندوق‌پستی به عنوان بخشی از فضای آدرس پردازه به حساب بیاید)، در این صورت میان مالک (تنها کسی که می‌تواند از این صندوق‌پستی پیغام دریافت کند) و کاربر آن (کسی که فقط می‌تواند به صندوق‌پستی پیغام بگذارد) تفاوت قایل می‌شویم. هر زمان که پردازه‌ی مالک صندوق‌پستی خاتمه می‌پذیرد، صندوق‌پستی از بین می‌رود. پس از آن، بایستی به هر پردازه‌ای که قصد دارد به این صندوق‌پستی پیغام بفرستد اطلاع داده شود که دیگر صندوق‌پستی وجود ندارد.

در مقایسه، یک صندوق‌پستی که تحت مالکیت سیستم‌عامل قرار دارد، موجودیت خاص خود را دارد. همین‌طور، مستقل بوده و به هیچ پردازه‌ی خاصی متصل نمی‌شود. در اینجا، سیستم‌عامل بایستی راهکاری فراهم نماید تا یک پردازه بتواند عملیات زیر را انجام بدهد:

- ایجاد یک صندوق‌پستی جدید.
- ارسال و دریافت پیغام‌ها از طریق صندوق‌پستی.
- حذف یک صندوق‌پستی.

پردازه‌ای که صندوق‌پستی جدیدی ایجاد می‌کند، مالک پیش‌فرض آن صندوق‌پستی است. در آغاز، مالک فقط پردازه‌ای است که می‌تواند از صندوق‌پستی پیغام دریافت کند. با این وجود، امتیاز مالکیت و دریافت صندوق می‌تواند از طریق فراخوانی‌های سیستمی مناسبی به سایر پردازه‌ها نیز اعطا شود. البته، پیامد این طرح آن است که هر صندوق‌پستی، گیرندگان متعددی خواهد داشت.

۳-۶-۲ همگام‌سازی

ارتباط میان پردازه‌ها از طریق فراخوانی‌های بدوی ()send و ()receive صورت می‌گیرد. گزینه‌های طراحی متفاوتی برای پیاده‌سازی هر یک از این عملیات بدوی وجود دارند. عبور پیغام می‌تواند **مسدودشونده**[1] و یا **بی‌انسداد**[2] باشد—که به **همگام**[3] و **ناهمگام**[4] هم معروف هستند. (در این کتاب، با مفاهیم رفتار همگام و ناهمگام در رابطه با الگوریتم‌های مختلف سیستم‌عامل مواجه خواهید شد.)

- **ارسال مسدودشونده**: پردازه‌ی فرستنده مسدود می‌شود تا این که، پیغام توسط پردازه‌ی گیرنده یا صندوق‌پستی دریافت شود.
- **ارسال بی‌انسداد**: پردازه‌ی فرستنده، پیغام را فرستاده و عملیات خود را از سر می‌گیرد.
- **دریافت مسدودشونده**: گیرنده مسدود می‌شود تا این که، یک پیغام دریافت کند.
- **دریافت بی‌انسداد**: گیرنده، یک پیغام معتبر و یا یک پیغام null دریافت می‌کند.

ترکیبات متفاوتی از ()send و ()receive امکان‌پذیرند. زمانی که ()send و ()receive هر دو مسدودشونده هستند، یک **ملاقات**[5] میان فرستنده و گیرنده خواهیم داشت. در صورتی که از دستورات ()send و ()receive مسدودشونده استفاده کنیم راه‌حل مسأله‌ی تولیدکننده–مصرف‌کننده بدیهی می‌شود. تولیدکننده کافی است فراخوانی ()send مسدودشونده را احضار و منتظر بماند تا پیغام به گیرنده یا صندوق‌پستی تحویل گردد. زمانی هم که مصرف‌کننده ()receive را احضار می‌کند، مسدود می‌شود تا این که پیغامی دریافت کند. این وضعیت در شکل‌های ۳-۱۴ و ۳-۱۵ به تصویر کشیده می‌شود.

۳-۶-۳ بافرسازی

فارغ از این که ارتباط مستقیم یا غیرمستقیم است، پیغام‌هایی که توسط پردازه‌های مرتبط مبادله می‌شوند در یک صف موقت قرار می‌گیرند. پیاده‌سازی این صف‌ها در اصل به سه روش زیر امکان‌پذیر است:

- **ظرفیت صفر**: حداکثر طول صف صفر است؛ بنابراین، خط ارتباطی هیچ پیغام منتظری نخواهد داشت. در این حالت، فرستنده بایستی مسدود شود تا این که، پیغام ارسالی توسط گیرنده دریافت شود.
- **ظرفیت محدود**[6]: طول صف، متناهی و برابر n است؛ بنابراین، حداکثر می‌توان تعداد n پیغام را در صف قرار داد. زمانی که پیغام جدیدی ارسال می‌شود، اگر صف پر نباشد، پیغام (کپی شده و یا اشاره‌گری به پیغام گذاشته می‌شود) در صف قرار داده می‌شود؛ لذا، فرستنده می‌تواند بدون انتظار اجرا شود. با این وجود، ظرفیت خط ارتباطی متناهی است. اگر خط ارتباطی پر باشد، فرستنده بایستی مسدود شود تا در

1. Blocking
2. Nonblocking
3. Synchronous
4. Asynchronous
5. Rendezvous
6. Bounded capacity

صف جا باز شود.

- **ظرفیت نامحدود**[1]: طول صف به صورت بالقوه نامتناهی است؛ بنابراین، پیغام‌ها به هر تعداد می‌توانند در آن منتظر شوند. فرستنده هرگز مسدود نمی‌شود.

```
message next_produced;

while (true) {
        /* produce an item in next_produced */

        send(next_produced);
}
```

شکل ۳-۱۴ پردازه‌ی تولیدکننده با استفاده از عبور پیغام.

```
message next_consumed;

while (true) {
        receive(next_consumed);

        /* consume the item in next_consumed */
}
```

شکل ۳-۱۵ پردازه‌ی مصرف‌کننده با استفاده از عبور پیغام.

۳-۷ مثال‌هایی از سیستم‌های IPC

در این بخش، چهار سیستم متفاوت IPC را بررسی می‌کنیم. در ابتدا API پوزیکس را برای حافظه‌ی مشترک مطرح کرده و سپس به بحث عبور پیغام در سیستم‌عامل Mach می‌پردازیم. در ادامه، IPC ویندوز را مطرح می‌کنیم که به طرز جالبی از حافظه‌ی مشترک به عنوان راهکاری برای فراهم‌نمودن انواع خاصی از عبور پیغام استفاده می‌کند. ما با لوله‌ها جمع‌بندی می‌کنیم که یکی از راهکارهای IPC نخستین در سیستم‌های یونیکس می‌باشد.

۳-۷-۱ حافظه‌ی مشترک پوزیکس

چندین راهکار IPC برای سیستم‌های پوزیکس موجودند که از آن جمله می‌توان به حافظه‌ی مشترک و عبور پیغام اشاره کرد. در اینجا، به حافظه‌ی مشترک API پوزیکس می‌پردازیم.

حافظه‌ی مشترک پوزیکس با استفاده از فایل‌های نگاشته به حافظه که ناحیه‌ی حافظه‌ی مشترک را به یک فایل مرتبط می‌کند، سازمان‌دهی می‌شود. هر پردازه در ابتدا بایستی با استفاده از فراخوانی سیستمی shm_open()، یک شئ حافظه‌ی مشترک را به صورت زیر ایجاد کند:

```
fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

1. Unbounded capacity